Heart disease decision tree

April 26, 2026

Karolina Antonowicz, Pb 117296, grupa A
Heart Disease Classification – Decision Tree

Celem projektu jest analiza zbioru Heart Disease oraz budowa modelu klasyfikacyjnego z wykorzystaniem drzewa decyzyjnego. Zbiór danych pochodzi z bazy UCI Machine Learning Repository (Cleveland dataset). Zawiera 303 instancje opisane przez 14 atrybutów klinicznych.

1. Importowanie bibliotek

W tej części kodu wczytywane są narzędzia niezbędne do pracy. Biblioteki pandas i numpy służą do manipulacji danymi, matplotlib i seaborn odpowiadają za wykresy, a pakiety z sklearn dostarczają algorytm drzewa decyzyjnego oraz funkcje do podziału danych i oceny jakości modelu.

[20]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import classification_report, roc_curve, auc
```

2. Wczytanie danych

Kod definiuje listę nazw kolumn, ponieważ plik źródłowy ich nie posiada, a następnie wczytuje dane z pliku tekstowego do struktury DataFrame. Wywołanie metody .head() pozwala podejrzeć pierwsze pięć wierszy zbioru, aby upewnić się, że dane zostały poprawnie zaimportowane.

[21]:
```
columns = [
    "age", "sex", "cp", "trestbps", "chol", "fbs",
    "restecg", "thalach", "exang", "oldpeak",
    "slope", "ca", "thal", "target"
]

df = pd.read_csv("data/processed.cleveland.data", names=columns)
df.head()
```

[21]:
```
    age  sex   cp  trestbps   chol  fbs  restecg  thalach  exang  oldpeak  \
0  63.0  1.0  1.0     145.0  233.0  1.0      2.0    150.0    0.0      2.3
1  67.0  1.0  4.0     160.0  286.0  0.0      2.0    108.0    1.0      1.5
2  67.0  1.0  4.0     120.0  229.0  0.0      2.0    129.0    1.0      2.6
```

```
3  37.0  1.0  3.0     130.0  250.0  0.0      0.0    187.0    0.0      3.5
4  41.0  0.0  2.0     130.0  204.0  0.0      2.0    172.0    0.0      1.4

   slope   ca thal  target
0    3.0  0.0  6.0       0
1    2.0  3.0  3.0       2
2    2.0  2.0  7.0       1
3    3.0  0.0  3.0       0
4    1.0  0.0  3.0       0
```

3. Wstępna anliza danych

Za pomocą funkcji .info() sprawdzane są typy danych w kolumnach oraz wykrywane są kolumny o formacie tekstowym. Metoda .describe() generuje statystyki opisowe, takie jak średnia, odchylenie standardowe czy wartości skrajne dla wszystkich cech numerycznych.

[22]:
```
df.info()
df.describe()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       303 non-null    float64
 1   sex       303 non-null    float64
 2   cp        303 non-null    float64
 3   trestbps  303 non-null    float64
 4   chol      303 non-null    float64
 5   fbs       303 non-null    float64
 6   restecg   303 non-null    float64
 7   thalach   303 non-null    float64
 8   exang     303 non-null    float64
 9   oldpeak   303 non-null    float64
 10  slope     303 non-null    float64
 11  ca        303 non-null    str
 12  thal      303 non-null    str
 13  target    303 non-null    int64
dtypes: float64(11), int64(1), str(2)
memory usage: 33.3 KB
```

[22]:
```
              age         sex          cp    trestbps        chol         fbs  \
count  303.000000  303.000000  303.000000  303.000000  303.000000  303.000000
mean    54.438944    0.679868    3.158416  131.689769  246.693069    0.148515
std      9.038662    0.467299    0.960126   17.599748   51.776918    0.356198
min     29.000000    0.000000    1.000000   94.000000  126.000000    0.000000
25%     48.000000    0.000000    3.000000  120.000000  211.000000    0.000000
50%     56.000000    1.000000    3.000000  130.000000  241.000000    0.000000
75%     61.000000    1.000000    4.000000  140.000000  275.000000    0.000000
```

```
max     77.000000    1.000000    4.000000  200.000000  564.000000    1.000000

          restecg     thalach       exang     oldpeak       slope      target
count  303.000000  303.000000  303.000000  303.000000  303.000000  303.000000
mean     0.990099  149.607261    0.326733    1.039604    1.600660    0.937294
std      0.994971   22.875003    0.469794    1.161075    0.616226    1.228536
min      0.000000   71.000000    0.000000    0.000000    1.000000    0.000000
25%      0.000000  133.500000    0.000000    0.000000    1.000000    0.000000
50%      1.000000  153.000000    0.000000    0.800000    2.000000    0.000000
75%      2.000000  166.000000    1.000000    1.600000    2.000000    2.000000
max      2.000000  202.000000    1.000000    6.200000    3.000000    4.000000
```

Brakujace wartości
Program wyszukuje znaki zapytania ("?"), które w tym zbiorze oznaczają brak danych, i zamienia je na standardowy format NaN. Następnie wszystkie dane są konwertowane na typ numeryczny, co pozwala na dalsze obliczenia i identyfikację braków w kolumnach takich jak ca czy thal.

[23]:
```
df.replace("?", np.nan, inplace=True)
df = df.apply(pd.to_numeric)
df.isnull().sum()
```

[23]:
```
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          4
thal        2
target      0
dtype: int64
```

Obsługa braku danych
Zamiast usuwać wiersze z brakami, kod uzupełnia puste miejsca wartością mediany dla danej kolumny za pomocą funkcji .fillna(). Jest to bezpieczna technika, która pozwala zachować pełną liczebność zbioru przy jednoczesnym zminimalizowaniu wpływu wartości odstających na model.

[24]:
```
df.fillna(df.median(), inplace=True)
```

[24]:
```
      age  sex   cp  trestbps   chol  fbs  restecg  thalach  exang  oldpeak  \
0    63.0  1.0  1.0     145.0  233.0  1.0      2.0    150.0    0.0      2.3
1    67.0  1.0  4.0     160.0  286.0  0.0      2.0    108.0    1.0      1.5
2    67.0  1.0  4.0     120.0  229.0  0.0      2.0    129.0    1.0      2.6
```

```
3    37.0  1.0  3.0     130.0  250.0  0.0      0.0    187.0    0.0      3.5
4    41.0  0.0  2.0     130.0  204.0  0.0      2.0    172.0    0.0      1.4
..    …   …   …       …    …   …       …       …      …
298  45.0  1.0  1.0     110.0  264.0  0.0      0.0    132.0    0.0      1.2
299  68.0  1.0  4.0     144.0  193.0  1.0      0.0    141.0    0.0      3.4
300  57.0  1.0  4.0     130.0  131.0  0.0      0.0    115.0    1.0      1.2
301  57.0  0.0  2.0     130.0  236.0  0.0      2.0    174.0    0.0      0.0
302  38.0  1.0  3.0     138.0  175.0  0.0      0.0    173.0    0.0      0.0

     slope   ca  thal  target
0      3.0  0.0   6.0       0
1      2.0  3.0   3.0       2
2      2.0  2.0   7.0       1
3      3.0  0.0   3.0       0
4      1.0  0.0   3.0       0
..      …   …   …     …
298    2.0  0.0   7.0       1
299    2.0  2.0   7.0       2
300    2.0  1.0   7.0       3
301    2.0  1.0   3.0       1
302    1.0  0.0   3.0       0

[303 rows x 14 columns]
```

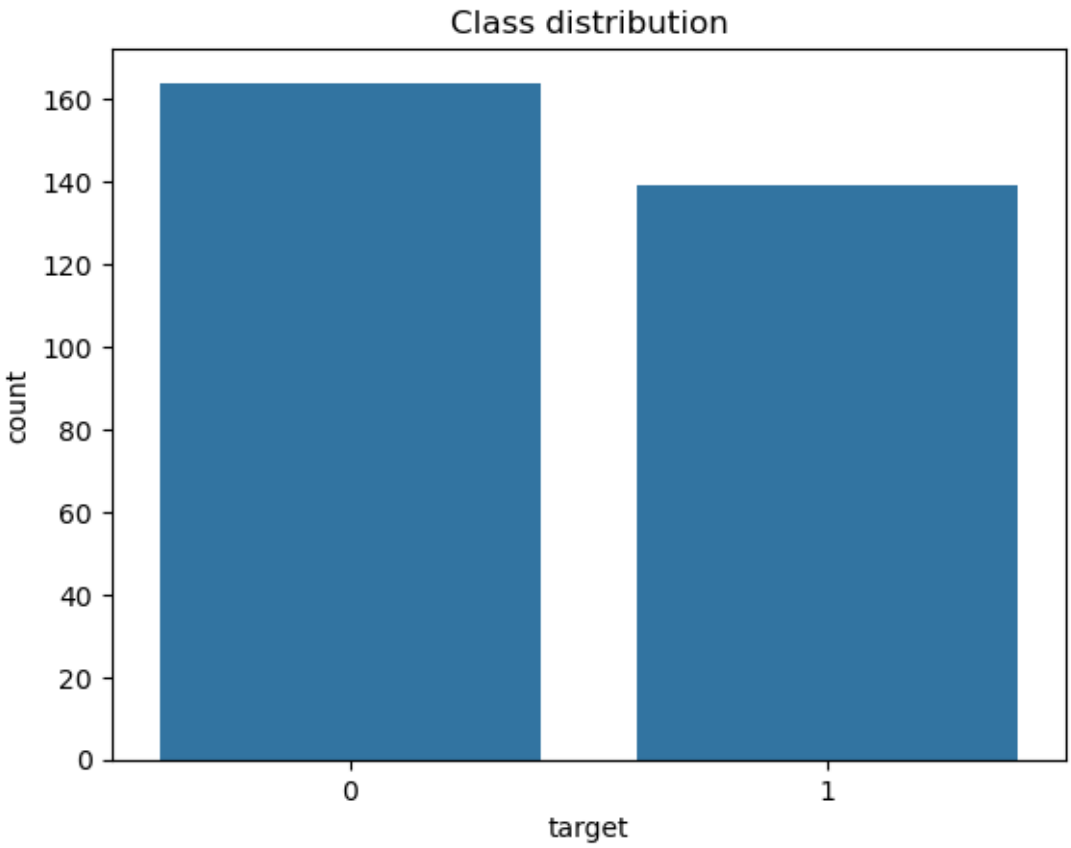
Przygotowanie zmiennej celu (target)
Oryginalna zmienna target zawierała wartości od 0 do 4, wskazujące na stopień zaawansowania choroby. Kod przekształca to w problem klasyfikacji binarnej: wartość 0 pozostaje jako brak choroby, natomiast wszystkie wartości powyżej 0 są zamieniane na 1, co oznacza obecność schorzenia.

[25]:
```
df['target'] = df['target'].apply(lambda x: 1 if x > 0 else 0)
```

5. Wizualizacja danych

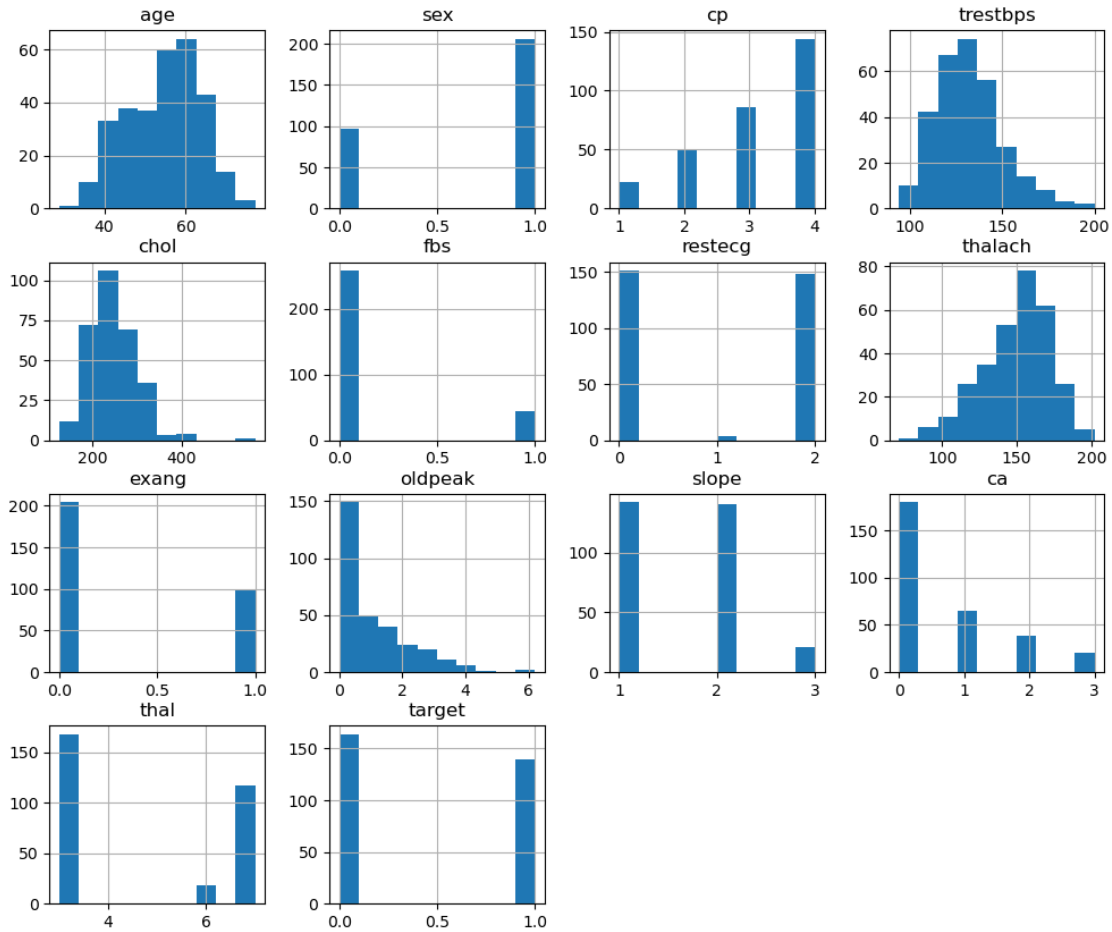
Przy użyciu biblioteki seaborn generowany jest wykres słupkowy, który pokazuje liczebność osób zdrowych względem chorych. Pozwala to ocenić, czy zbiór danych jest zbalansowany, co ma kluczowe znaczenie dla rzetelności wyników klasyfikacji.

[26]:
```
sns.countplot(x='target', data=df)
plt.title("Class distribution")
plt.show()
```



Historgramy
Ta sekcja generuje zestaw histogramów dla wszystkich atrybutów w zbiorze danych. Dzięki temu można zaobserwować rozkłady poszczególnych cech i zidentyfikować ewentualne anomalie lub specyficzne tendencje w populacji pacjentów.

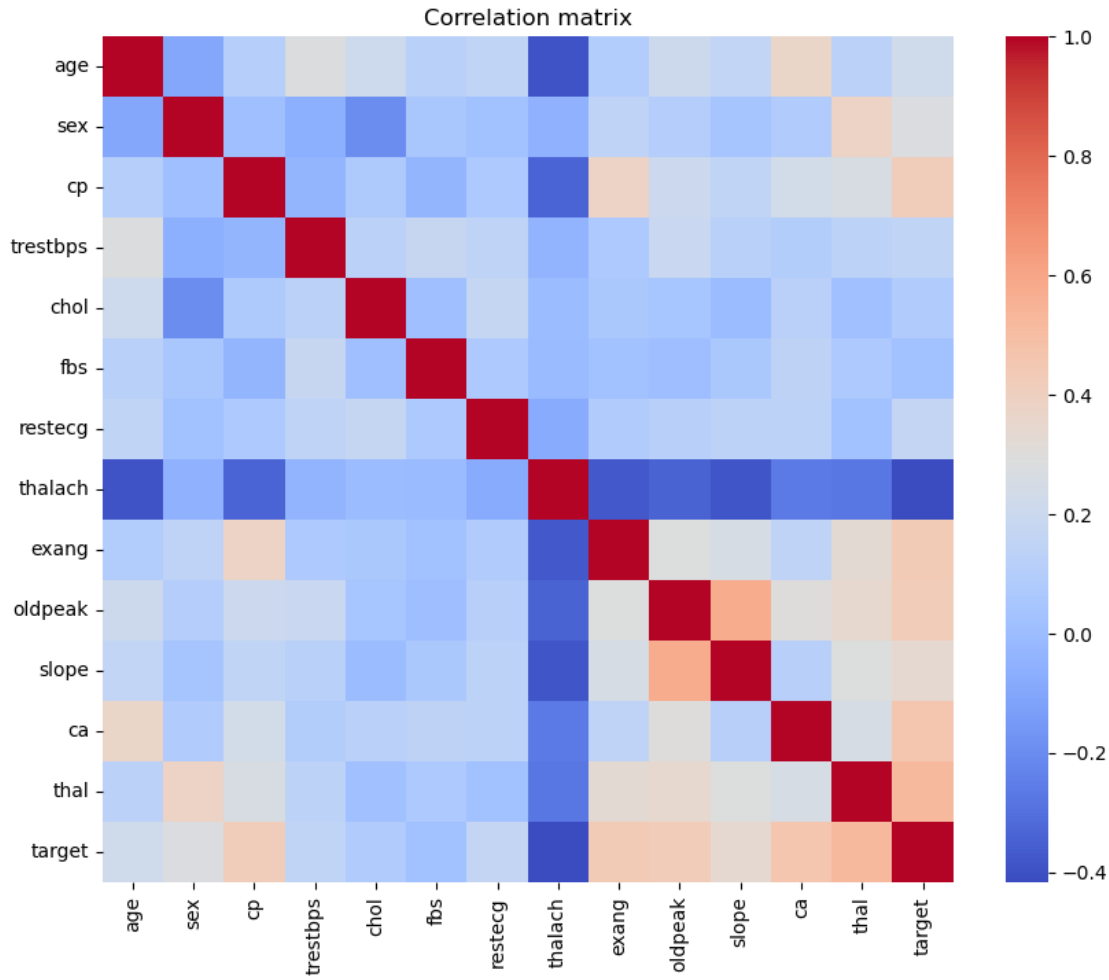
[27]:
```
df.hist(figsize=(12,10))
plt.show()
```



Macierz korelacji

Ten kod generuje mapę ciepła (heatmapę), która wizualizuje macierz korelacji, pozwalając szybko zidentyfikować siłę i kierunek powiązań między wszystkimi cechami w zbiorze danych. Kolory na wykresie pomagają wykryć zmienne, które są silnie ze sobą powiązane, co jest kluczowe przy wyborze cech mających największy wpływ na występowanie chorób serca.

[28]:
```
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), cmap='coolwarm')
plt.title("Correlation matrix")
plt.show()
```



6. Podział danych

Kod wydziela kolumnę target jako etykietę (y) i pozostałe cechy jako dane wejściowe (X), a następnie dzieli je w proporcji 80/20. Zbiór treningowy służy do budowy drzewa, a zbiór testowy do sprawdzenia, jak model poradzi sobie z nowymi, nieznanymi wcześniej przypadkami.

[29]:
```
X = df.drop('target', axis=1)
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

7. Model bazowy

W tym miejscu dokonujemy wstępnej oceny modelu drzewa decyzyjnego przy użyciu domyślnych parametrów. Generujemy raport klasyfikacji, który pokazuje precyzję i czułość algorytmu, co pozwala nam sprawdzić, jak model radzi sobie z diagnozą choroby serca przed przystąpieniem do jego optymalizacji.

[30]:
```
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

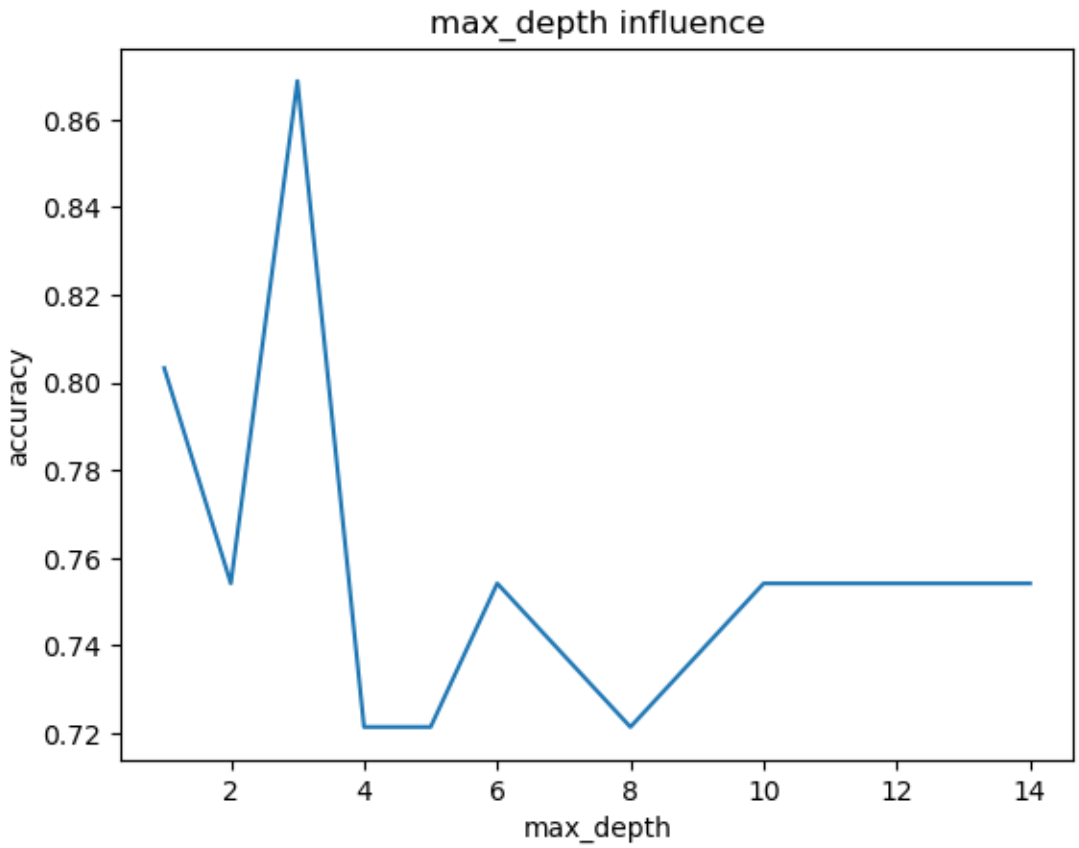
[31]:
```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.73      0.76      0.75        29
           1       0.77      0.75      0.76        32

    accuracy                           0.75        61
   macro avg       0.75      0.75      0.75        61
weighted avg       0.75      0.75      0.75        61
```

8. Eksperyment - wpływ max_depth

W tej części kodu tworzymy podstawowy klasyfikator drzewa decyzyjnego i trenujemy go na przygotowanych danych za pomocą funkcji .fit(), co pozwala algorytmowi wygenerować reguły decyzyjne. Następnie używamy funkcji classification_report, aby otrzymać wstępny raport skuteczności, który pokazuje, jak dobrze model radzi sobie z rozpoznawaniem chorych i zdrowych pacjentów przed wprowadzeniem jakichkolwiek ulepszeń. Służy to jako punkt odniesienia, dzięki któremu w następnym kroku będziemy mogli ocenić, czy optymalizacja za pomocą GridSearchCV faktycznie poprawiła wyniki.

[32]:
```
depths = range(1, 15)
scores = []

for d in depths:
    m = DecisionTreeClassifier(max_depth=d, random_state=42)
    m.fit(X_train, y_train)
    scores.append(m.score(X_test, y_test))

plt.plot(depths, scores)
plt.xlabel("max_depth")
plt.ylabel("accuracy")
plt.title("max_depth influence")
plt.show()
```



9. Grid search

W tej sekcji definiujemy param_grip, czyli listę specjalnych ustawień. Dla poprawy wynikiów modelu bazowego, stosujemy procedurę GridSearchCV, która automatycznie przeszukuje różne ustawienia modelu, takie jak maksymalna głębokość drzewa. Algorytm testuj/e wiele kombinacji i wybiera tę, która najlepiej radzi sobie z danymi, co pozwala zminimalizować ryzyko błędu i poprawić zdolność modelu do ogólnego wnioskowania o zdrowiu pacjentów.

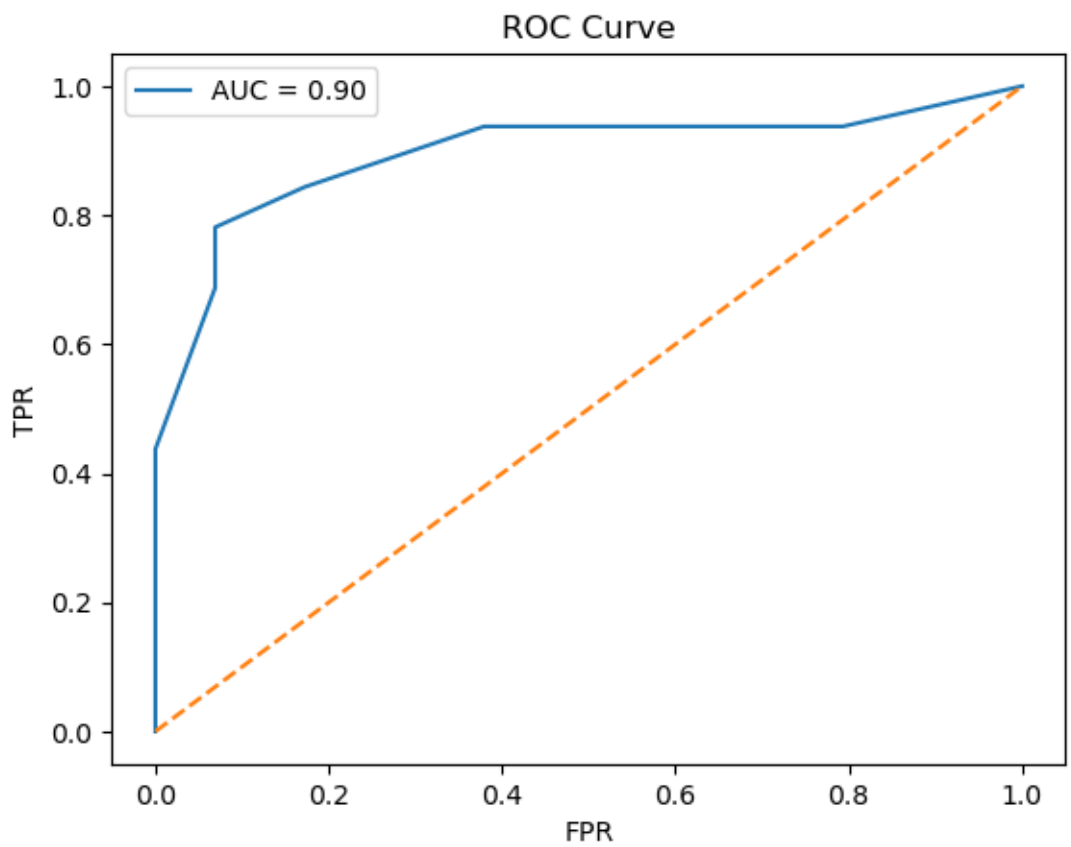
[33]:
```
params = {
    'max_depth': [3, 5, 7, 10],
    'min_samples_split': [2, 5, 10],
    'criterion': ['gini', 'entropy']
}

grid = GridSearchCV(DecisionTreeClassifier(), params, cv=5)
grid.fit(X_train, y_train)

print("Najlepsze parametry:", grid.best_params_)
```

```
Najlepsze parametry: {'criterion': 'entropy', 'max_depth': 3,
'min_samples_split': 2}
```

10. ROC curve

Kod oblicza i rysuje krzywą ROC, która ilustruje jakość klasyfikacji przy różnych progach odcięcia. Wartość AUC (Area Under Curve) podana na wykresie informuje o ogólnej zdolności modelu do odróżniania klas – im bliżej 1.0, tym lepszy jest model.

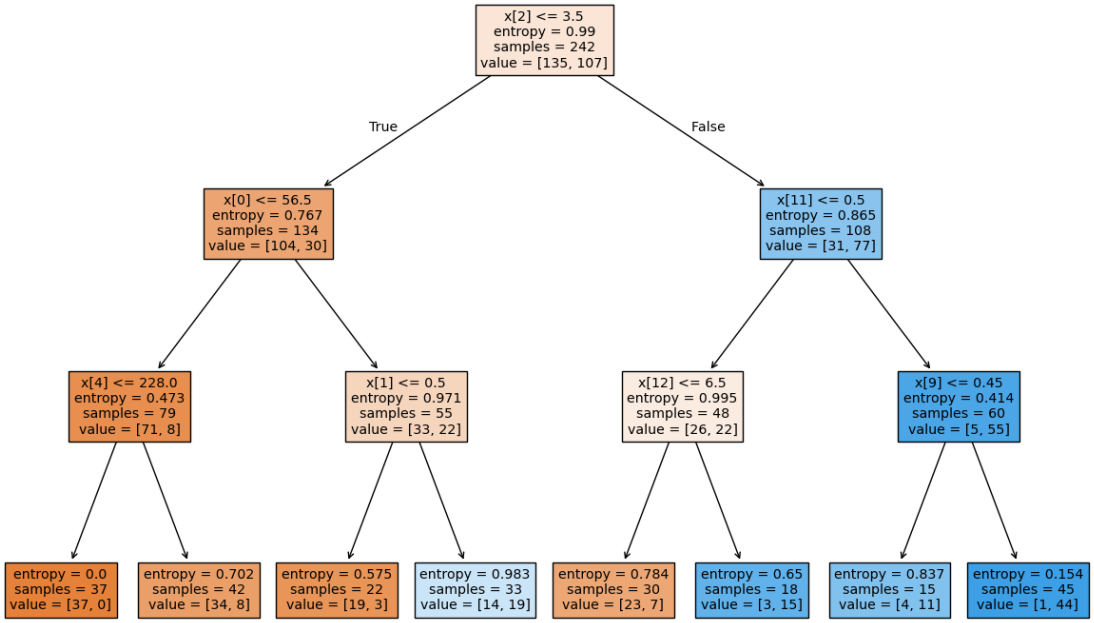
[34]:
```
best_model = grid.best_estimator_
y_prob = best_model.predict_proba(X_test)[:,1]

fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.2f}")
plt.plot([0,1],[0,1],'--')
plt.xlabel("FPR")
plt.ylabel("TPR")
plt.title("ROC Curve")
plt.legend()
plt.show()
```



11. Wizualizacja drzewa

Funkcja plot_tree generuje graficzny schemat decyzji podejmowanych przez algorytm. Możemy prześledzić krok po kroku, jakie warunki prowadzą do finalnej diagnozy, co czyni model w pełni zrozumiałym dla człowieka.

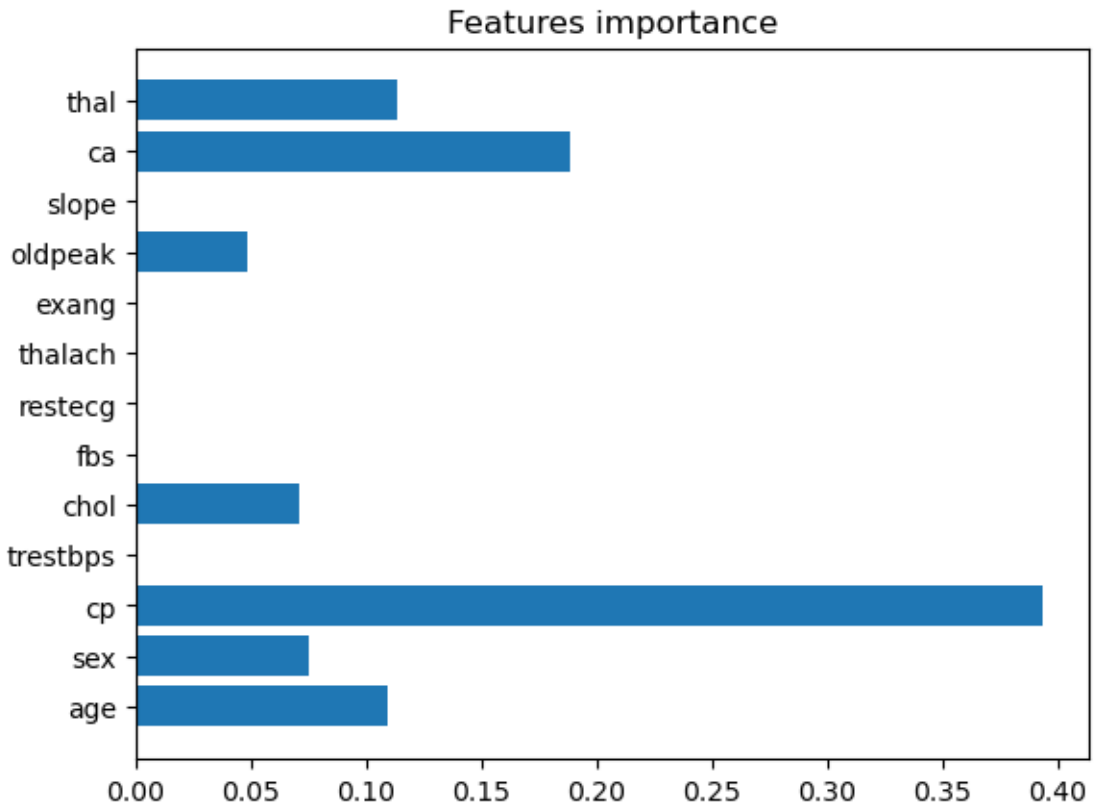
[35]:
```
plt.figure(figsize=(15,10))
plot_tree(best_model, filled=True)
plt.show()
```



12. Feature importance

Ostatni fragment kodu tworzy wykres słupkowy pokazujący, które cechy miały największy wpływ na wynik klasyfikacji. Analiza ta pozwala zidentyfikować kluczowe wskaźniki medyczne, które w największym stopniu determinują ryzyko wystąpienia choroby serca w tym zbiorze.

[36]:
```
importance = best_model.feature_importances_

plt.barh(X.columns, importance)
plt.title("Features importance")
plt.show()
```



13. Wnioski

Model drzewa decyzyjnego skutecznie klasyfikuje występowanie choroby serca, osiągając dobrą jakość predykcji potwierdzoną metrykami oraz krzywą ROC. Odpowiedni dobór hiperparametrów (szczególnie ograniczenie głębokości drzewa) pozwala uniknąć przeuczenia i poprawia zdolność generalizacji modelu. Analiza ważności cech wskazuje, że wybrane parametry kliniczne mają istotny wpływ na decyzje modelu. Mimo dobrych wyników, model można ulepszyć poprzez zastosowanie metod zespołowych oraz bardziej zaawansowanych technik imputacji braków danych.